Deep Learning

Module 2: Training Multilayer Perceptrons

Quick Reference Guide

Learning Outcomes:

1. Compare and contrast popular activation functions.
2. Define vector activations as neurons that produce multiple outputs.
3. Explain output representations for binary and multiclass classification problems.
4. Describe the purpose and significance of the divergence term in the network loss function.
5. Set up variables to compute the derivative of a single training input.
6. Order the steps to calculate the output of the network from input (forward pass).
7. Explain the chain rule of calculus in the context of neural networks.
8. Order the steps of backpropagation.
9. Order the steps of the forward pass using vector notation.
10. Explain the chain rule for vector functions in the context of neural networks.
11. Order the steps of backpropagation using vector notation.
12. Summarize the forward and backward pass algorithms for neural networks.
13. Implement the forward pass, backward pass, and stochastic gradient descent for a multilayer perceptron.

Introduction to the Learning Problem

Given a training set of input–output pairs, $(X_1, d_1), (X_2, d_2), \ldots, (X_T, d_T)$, minimize the function

$$\text{Loss}(W) = \frac{1}{T}\sum_i div(f(X_i, W_i), d_i).$$

Here f is an activation function. Examine different activation functions and their derivatives.

Sigmoid activation: $f(z) = \frac{1}{1 + exp(-z)}$; $f'(z) = f(z)\big(1 - f(z)\big)$

Tanh activation: $f(z) = \tanh(z)$; $f'(z) = 1 - f^2(z)$

ReLU activation: $f(z) = \begin{cases} z, z \geq 0 \\ 0, z < 0 \end{cases}$; $f'(z) = \begin{cases} 1, z \geq 0 \\ 0, z < 0 \end{cases}$

Softplus activation: $f(z) = \log(1 + exp(-z))$; $f'(z) = \frac{1}{1 + exp(-z)}$

Problem Setup

Neurons that produce multiple outputs are called vector activations.

$[y_1, y_2, \ldots, y_l] = f(x_1, x_2, \ldots, x_k; W)$

Consider the affine term $z_j = \sum_j w_{ji} x_j + b_i$, where w_{ji} are weights and b_i are biases.

The output of a softmax vector activation is $y_i = \frac{exp(z_j)}{\sum_j exp(z_j)}$.

The derivatives of the softmax are $\frac{\partial y_i}{\partial z_i} = y_i(1 - y_i)$ and $\frac{\partial y_j}{\partial z_i} = -y_i y_j;\ for\ j \neq i$.

Derivatives and Gradients

For a multiclass classifier with *N* classes, the output will be an *N*-component probability vector. A softmax activation is often used at the output of multiclass classifier networks.

Affine terms:

$$z_i = \sum_j w_{ji}^{(n)} x_j^{(n-1)} + b_j$$

Outputs:

$$y_i = \frac{exp(z_j)}{\sum_j exp(z_j)} = P(class = i|X)$$

Function Minimization

The divergence function quantifies the difference between the actual and desired outputs of a network.

$div(f(X_i, W_i), d_i)$

To minimize the loss with respect to the parameters W using gradient descent, the divergence function must be continuous, and ideally, differentiable.

$$\text{Loss}(W) = \frac{1}{T}\sum_i div(f(X_i, W_i), d_i)$$

For regression models, we compute the L2 divergence.

$$\nabla_z \frac{1}{2}\|y - d\|^2 = (y - d)^T$$

For classification models, we compute the KL divergence or cross entropy loss.

$$\nabla_z KL(y, d) = (y - d)^T$$

Training a Neural Network

If $Y_i = f(X_i, W_i)$, the total training loss is:

$$\text{Loss}(W) = \frac{1}{T}\sum_i div(Y_i, d_i)$$

The weight of the connection between the i^{th} unit of the $(l-1)^{th}$ layer and the j^{th} unit of the l^{th} layer is ${W_{i,j}}^{(l)}$.

The total derivative is given by:

$$\frac{d\text{Loss}(W)}{d{W_{i,j}}^{(l)}} = \frac{1}{T}\sum_i \frac{ddiv(Y_i, d_i)}{d{W_{i,j}}^{(l)}}$$

We need to compute the derivative of a single training input $\frac{ddiv(Y_i,d_i)}{d{W_{i,j}}^{(l)}}$.

The Forward Pass

The derivative of the divergence depends on the intermediate values and the output of the network computed during the forward pass.

Here is the pseudocode for the forward pass.

- Input: D-dimensional vector $\boldsymbol{x} = [x_j, j = 1 \ldots D]$
- Set:
  - $D_0 = D$, the width of the 0^{th} input layer
  - ${y_j}^{(0)} = x_j, j = 1 \ldots D$; ${y_0}^{(k\,=\,1\,\ldots\,N)} = x_0 = 1$
- For layer $k = 1 \ldots N$:
  - For $j = 1 \ldots D_k$; # D_k is the size of the k^{th} layer:
    - ${z_j}^{(k)} = \sum_{i\,=\,0}^{D_k\,-\,1} {w_{ij}}^{(k)} {y_i}^{(k\,-\,1)}$
    - ${y_j}^{(k)} = f_k\left({z_j}^{(k)}\right)$
- Output:
  - $Y = {y_j}^{(N)}; \; j = 1 \ldots D_N$

Calculus Refresher: The Chain Rule

Consider this influence diagram of a neural network.

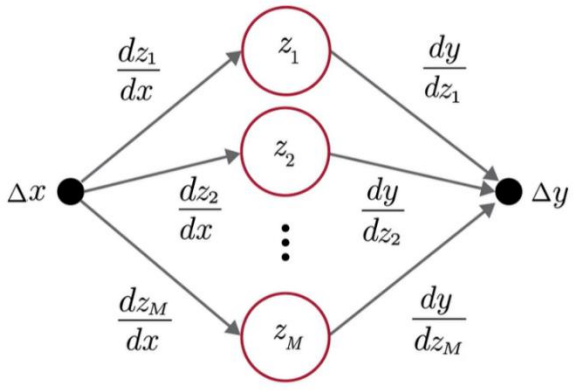

Figure 1: An influence diagram of a neural network with *M* input edges, *M* hidden layer neurons, and *M* output edges from the hidden layer neuron, where the derivatives with respect to the input layer and hidden layer neurons are marked as the weights to the respective edges

The distributed chain rule is defined as:

For $y = f(z_1, z_2, \ldots, z_M)$ where $z_i = g_i(x)$:

$$\Delta y = \sum_i \frac{\partial y}{\partial z_i} \Delta z_i$$

$$\frac{dy}{dx} = \frac{\partial y}{\partial z_1}\frac{\partial z_1}{\partial x} + \frac{\partial y}{\partial z_2}\frac{\partial z_2}{\partial x} + \cdots + \frac{\partial y}{\partial z_M}\frac{\partial z_M}{\partial x}$$

$$\Delta y = \sum_i \frac{\partial y}{\partial z_i}\frac{\partial z_i}{\partial x} \Delta x$$

So, Δy has additive contributions from each of the incoming paths. It is the sum of the product of the weights of all the incoming edges times Δx.

The Backward Pass

In the backward pass, we compute the derivatives from the intermediate values computed during the forward pass. We begin with computing the derivate of the divergence between the actual and desired outputs, and then move backward through the network to compute the rest of the derivatives. This is also called backpropagation.

Here is the pseudocode for the backward pass.

- Output layer (N):
  - For $i = 1 \ldots D_N$
    - $\frac{\partial Div}{\partial y_i^{(N)}} = \frac{\partial Div(Y, d)}{\partial y_i}$
    - $\frac{\partial Div}{\partial z_i^{(N)}} = \frac{\partial Div}{\partial y_i^{(N)}} f_N'\left(z_i^{(N)}\right)$
    - $\frac{\partial Div}{\partial w_{ij}^{(N)}} = y_i^{(N-1)} \frac{\partial Div}{\partial z_j^{(N)}}$ for $j = 1 \ldots D_{N-1}$

- For layer $k = N - 1$ downto 1
  - For $i = 1 \ldots D_k$
    - $\frac{\partial Div}{\partial y_i^{(k)}} = \sum_j w_{ij}^{(k+1)} \frac{\partial Div}{\partial z_j^{(k+1)}}$
    - $\frac{\partial Div}{\partial z_i^{(k)}} = \frac{\partial Div}{\partial y_i^{(k)}} f_k'\left(z_i^{(k)}\right)$
    - $\frac{\partial Div}{\partial w_{ij}^{(k)}} = y_i^{(k-1)} \frac{\partial Div}{\partial z_j^{(k)}}$ for $j = 1 \ldots D_{k-1}$

Vector Formulation of the Forward Pass

You have seen how to compute the forward pass term by term. Vectorized operations can make operations much simpler and faster.

Here is the vectorized version of the forward pass.

- Initialize $\boldsymbol{y}_0 = \boldsymbol{x}$
- Iterate through layers:

- For layer $k = 1$ *to* N:
  - $\boldsymbol{z}_k = \boldsymbol{W}_k \boldsymbol{y}_{k-1} + \boldsymbol{b}_k$
  - $\boldsymbol{y}_k = \boldsymbol{f}_k(\boldsymbol{z}_k)$
- Output:
  - $\boldsymbol{Y} = \boldsymbol{y}_N$

Vector Calculus

To vectorize operations in the backward pass, we need to define the chain rule for vector activations, affine functions, and parameters of a neural network.

1. The chain rule for activations:

   If $y = f(z)$, $\nabla_z Div = \nabla_y Div \nabla_z y$

   $$\nabla_z Div = \nabla_y Div J_y(z)$$

2. The chain rule at affine layers:

   $\nabla_{y_{k-1}} Div = \nabla_{z_k} Div \nabla_{y_{k-1}} z_k$

   But $z_k = W_k y_{k-1} + b_k \Rightarrow \nabla_{y_{k-1}} z_k = W_k$, therefore

   $\nabla_{y_{k-1}} Div = \nabla_{z_k} Div W_k$

3. The chain rule for parameters:

   $z_k = W_k y_{k-1} + b_k$

   $\nabla_{b_k} Div = \nabla_{z_k} Div$

   $\nabla_{W_k} Div = y_{k-1} \nabla_{z_k} Div$

Backpropagation Vector Notation

Here is the vectorized version of the backward pass.

- Set $\boldsymbol{y}_N = Y, \boldsymbol{y}_0 = \boldsymbol{x}$
- Initialize: Compute $\nabla_{\boldsymbol{y}_N} Div = \nabla_Y Div$
- For layer $k = N$ downto 1
  - Compute $J_{\boldsymbol{y}_k}(\boldsymbol{z}_k)$
  - Backward recursion step:
    - $\nabla_{\boldsymbol{z}_k} Div = \nabla_{\boldsymbol{y}_k} Div J_{\boldsymbol{y}_k}(\boldsymbol{z}_k)$
    - $\nabla_{\boldsymbol{y}_{k-1}} Div = \nabla_{\boldsymbol{z}_k} Div \, \boldsymbol{W}_k$
  - Gradient computation:
    - $\nabla_{\boldsymbol{W}_k} Div = \boldsymbol{y}_{k-1} \nabla_{\boldsymbol{z}_k} Div$
    - $\nabla_{\boldsymbol{b}_k} Div = \nabla_{\boldsymbol{z}_k} Div$

    - For layer $k = 1$ *to* N:
        - $\boldsymbol{z}_k = \boldsymbol{W}_k \boldsymbol{y}_{k-1} + \boldsymbol{b}_k$
        - $\boldsymbol{y}_k = \boldsymbol{f}_k(\boldsymbol{z}_k)$
- Output:
    - $\boldsymbol{Y} = \boldsymbol{y}_N$

Training by Backpropagation

Neural networks must be trained to minimize the average divergence between the network and desired outputs. Minimization is performed using gradient descent, where gradients can be computed by a forward pass inference followed by a backward pass.

Neural network training algorithm

- Initialize all weights and biases $(W_1, b_1, W_2, b_2, \ldots, W_N, b_N)$
- Do:
    - For all k, initialize $\nabla_{W_k} Loss = 0, \nabla_{b_k} Loss = 0$
    - For all $t = 1 : T$
        - (Forward pass) Compute:
            Output $Y(X_t)$
            Divergence $Div(Y_t, d_t)$
            $Loss \mathrel{+}= Div(Y_t, d_t)$
        - (Backward pass) For all k, compute:
            $\nabla_{y_k} Div = \nabla_{z_{k+1}} Div \, W_{k+1}$
            $\nabla_{z_k} Div = \nabla_{y_k} Div J_{y_k}(z_k)$
            $\nabla_{W_k} Div(Y_t, d_t) = y_{k-1} \nabla_{z_k} Div$
            $\nabla_{b_k} Div(Y_t, d_t) = \nabla_{z_k} Div$
            $\nabla_{W_k} Loss \mathrel{+}= \nabla_{W_k} Div(Y_t, d_t)$
            $\nabla_{b_k} Loss \mathrel{+}= \nabla_{b_k} Div(Y_t, d_t)$